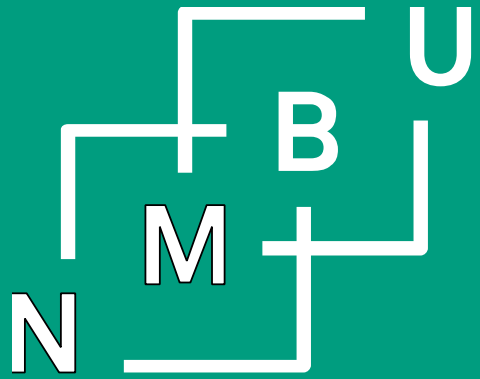


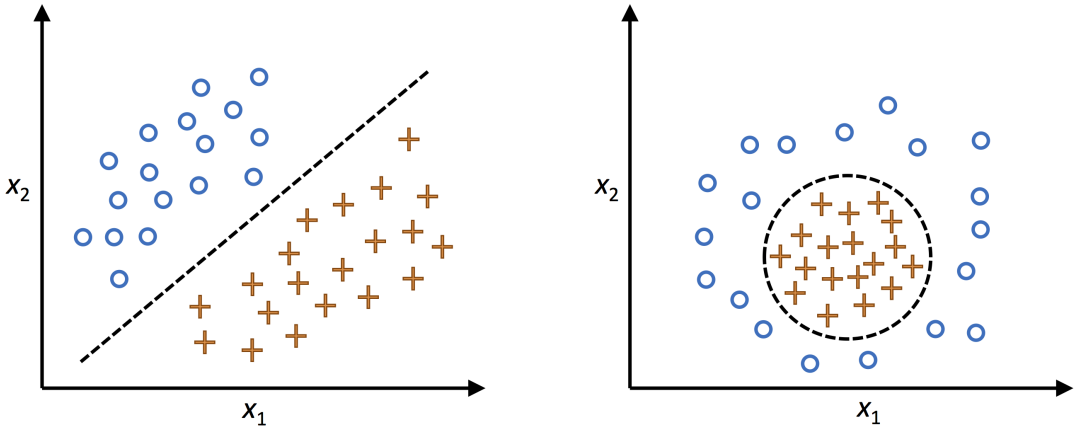
Norwegian University
of Life Sciences

Kernel PCA

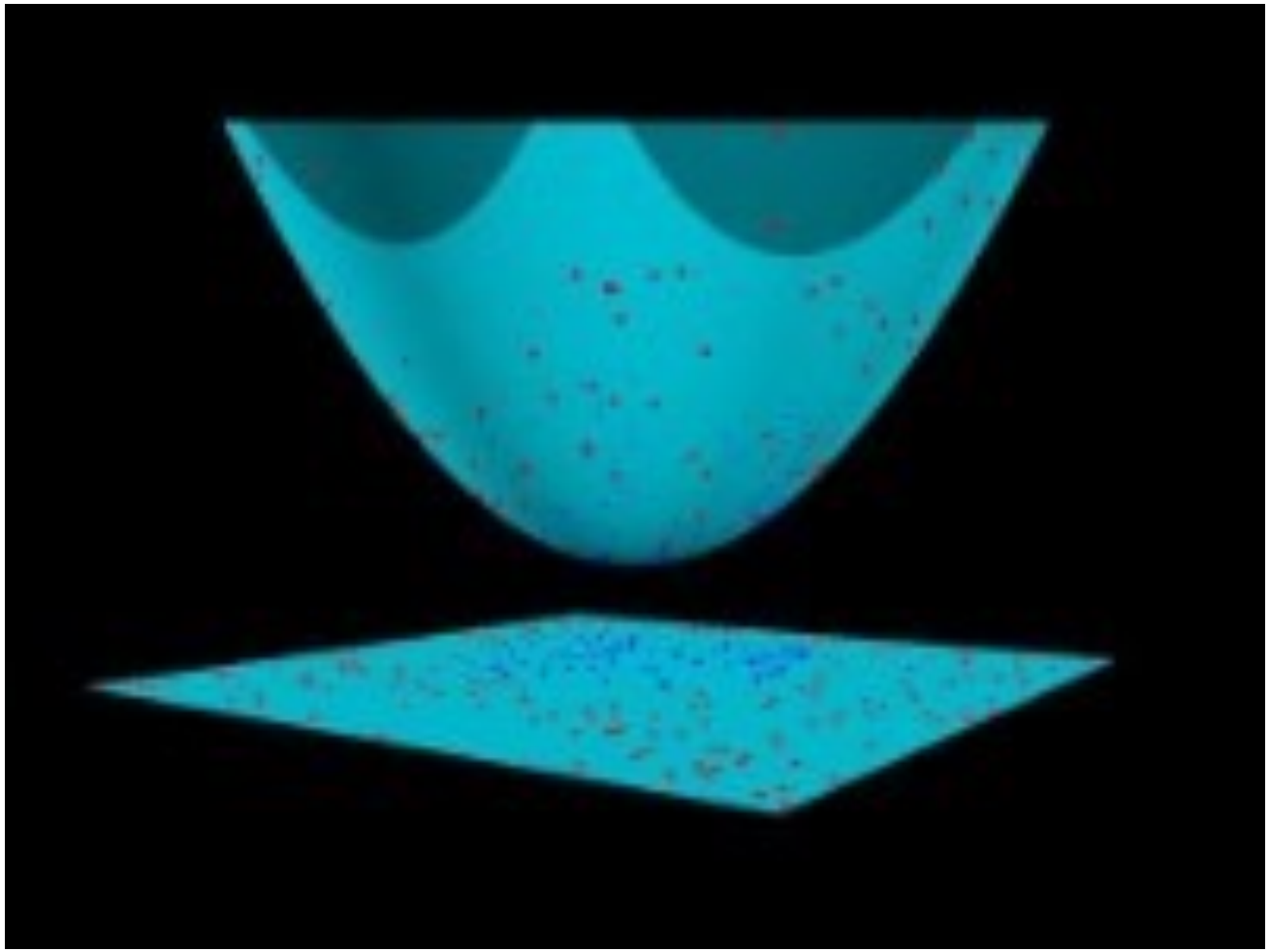
Chapter 5 in the Python Machine Learning book

Summary

- Kernels
  - Kernel PCA
  - The kernel trick
  - Various kernels



Intro



Source: https://www.youtube.com/watch?v=3liCbRZPrZA

Resources

- Python Machine Learning TE, Chapter 5, pages 169 – 189, (Chapter 3, pages 84 – 90)
  - Jupyter notebook: Chapter 5 - Kernels.ipynb

- PCA in scikit-learn: http://scikit-learn.org/stable/modules/decomposition.html#principal-component-analysis-pca
- Kernel SVM in scikit-learn: http://scikit-learn.org/stable/modules/svm.html#kernel-functions
- Kernel ridge regression in scikit-learn: http://scikit-learn.org/stable/modules/kernel_ridge.html

Kernels

- x → Original 2D input (feature vector)
- d → Dimensionality of the original space (e.g., 2)
- k → Dimensionality of the transformed space (e.g., 3)
- z → Transformed feature vector in higher-dimensional space
- ϕ → Mapping function to higher dimensions

- Nonlinear mapping to a higher dimensional space:

$$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^k \quad (k >> d)$$

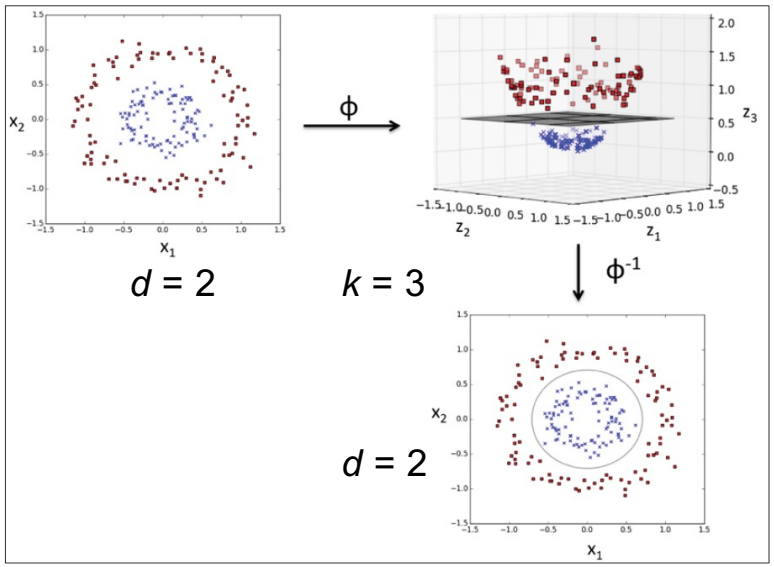
- Hopefully easier separation of classes / simpler relationship

$$\boldsymbol{x} = \left[x_1,\ x_2 \right]^T \qquad d = 2$$

$$\downarrow \phi$$

$$\mathbf{z} = \left[x_1^2, \sqrt{2x_1x_2}, x_2^2 \right]^T \qquad k = 3$$

$$\phi(x_1, x_2) = (z_1, z_2, z_3) = (x_1, x_2, x_1^2 + x_2^2)$$



Kernel Principal Component Analysis

- **Kernelized, nonlinear variant** of **PCA**
- **Non-zero eigenvalues** of $\boldsymbol{X^T X}$ and $\boldsymbol{X X^T}$ are the same

$$\boldsymbol{\left(X^T X\right)v = \lambda v} \quad \Longrightarrow \quad \boldsymbol{X\left(X^T X\right)v = \lambda X v} \quad \Longrightarrow \quad \boldsymbol{\left(X X^T\right)u = \lambda u;} \quad \boldsymbol{u := Xv}$$

- $\boldsymbol{X X^T}$ is the **kernel matrix** when using the **linear kernel function** $\boldsymbol{\kappa\left(x^{(i)}, x^{(j)}\right) = x^{(i)^T} x^{(j)}}$. It contains the inner products of all pairs of samples in the data set
- **Basic idea:** Replace $\boldsymbol{X X^T}$ with the kernel matrix of a nonlinear kernel function to perform PCA in some high-dimensional (possibly infinite dimensional) feature space
- The hope is that in this transformed feature space (nonlinear transformation) the data may be decomposable into linear combinations of the transformed features

Kernel Principal Component Analysis

- **Kernelized, nonlinear variant** of **PCA**
- **Non-zero eigenvalues** of $\boldsymbol{X^T X}$ and $\boldsymbol{X X^T}$ are the same

$$\boldsymbol{(X^T X)v = \lambda v} \quad \Longrightarrow \quad \boldsymbol{X(X^T X)v = \lambda X v} \quad \Longrightarrow \quad \boldsymbol{(X X^T)u = \lambda u;} \quad \boldsymbol{u := Xv}$$

- $\boldsymbol{X X^T}$ is the **kernel matrix** when using the **linear kernel function** $\boldsymbol{\kappa(x^{(i)}, x^{(j)}) = x^{(i)^T} x^{(j)}}$. It contains the inner products of all pairs of samples in the data set.

$$K_{ij} = \kappa(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \begin{bmatrix} \kappa(\mathbf{x}^{(1)}, \mathbf{x}^{(1)}) & \cdots & \kappa(\mathbf{x}^{(1)}, \mathbf{x}^{(n)}) \\ \vdots & \ddots & \vdots \\ \kappa(\mathbf{x}^{(n)}, \mathbf{x}^{(1)}) & \cdots & \kappa(\mathbf{x}^{(n)}, \mathbf{x}^{(n)}) \end{bmatrix}$$

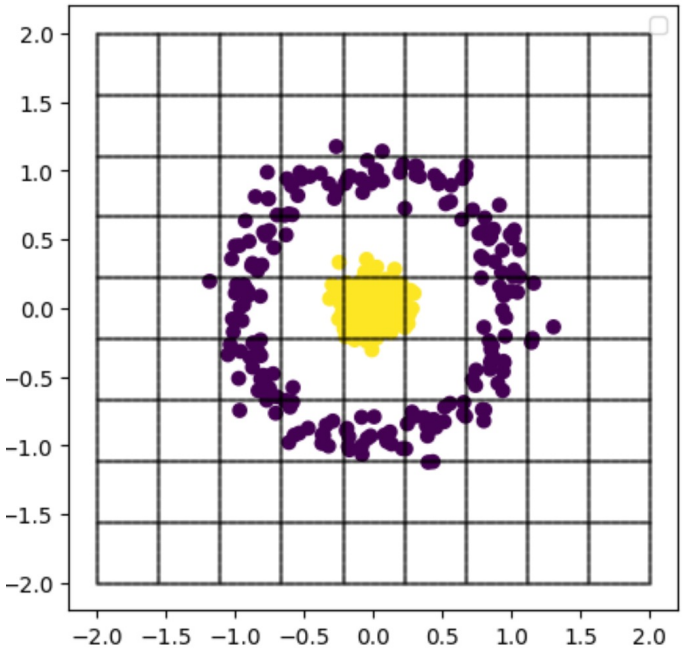
(Kernel matrix)

$$\kappa(\mathbf{x}, \mathbf{x}') : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$$

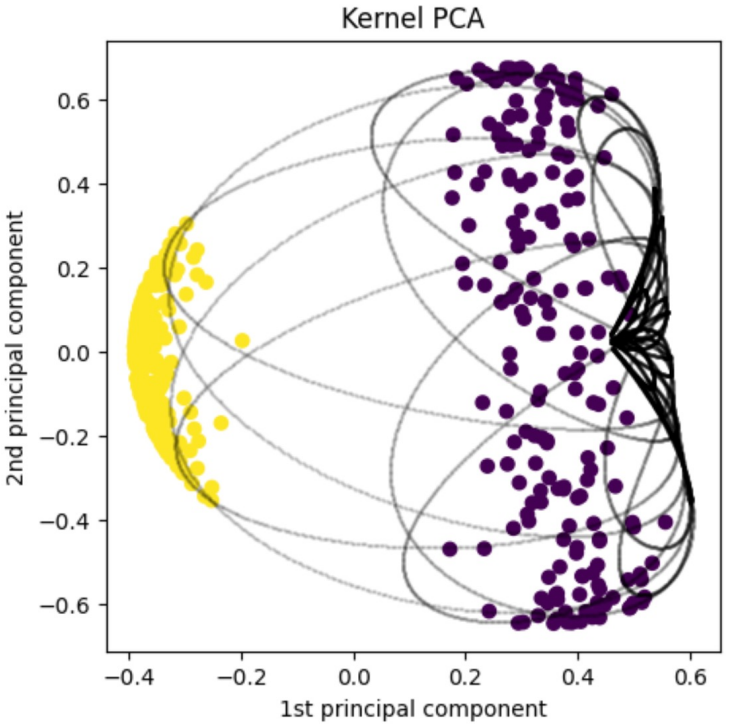
(Kernel function)

Kernel Principal Component Analysis

`05_KernelPCA.ipynb`



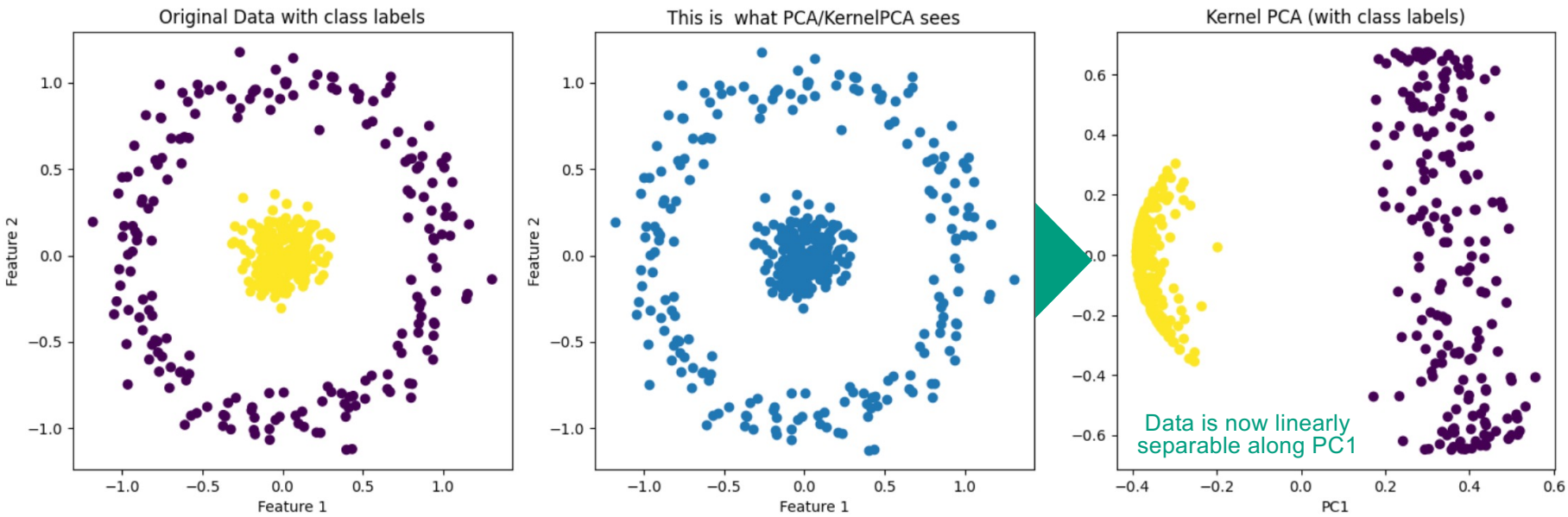
Kernel PCA



Kernel Principal Component Analysis

`05_KernelPCA.ipynb`

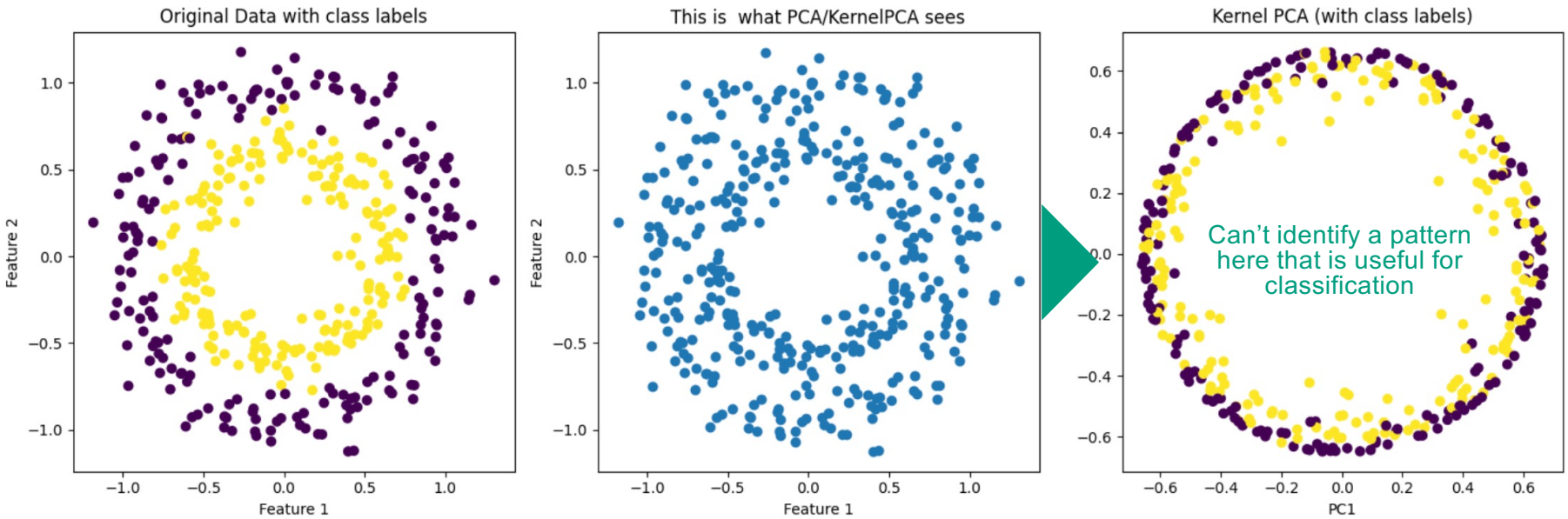
Be aware that PCA is an unsupervised learning technique! → Doesn't use labels



Kernel Principal Component Analysis

`05_KernelPCA.ipynb`

Be aware that PCA is an unsupervised learning technique! → Doesn't use labels



Kernel Principal Component Analysis

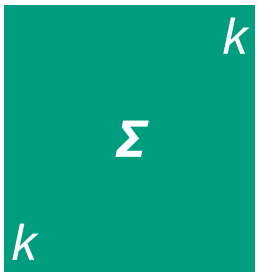
- Difficult to interpret
- Very sensitive to the kernel parameters
- Can identify possible nonlinear feature interactions / nonlinear patterns in the data
- Could be combined in a pipeline with a performant linear classifier

- Whether no PCA, linear, or kernel PCA is the best choice depends on the data set and the relationships between the features

Kernel PCA

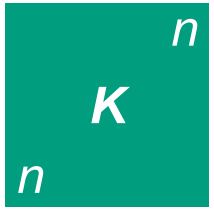
- Define $\boldsymbol{Z} = \phi(\boldsymbol{X})$ and the corresponding $\boldsymbol{\Sigma} = \frac{1}{n}\boldsymbol{Z}^t\boldsymbol{Z}$, where $\boldsymbol{Z}$ has dimension (n x k).





- By using the kernel matrix the eigendecomposition gets reduced from a (k x k)-problem to an (n x n)-problem:

$$\boldsymbol{K} = \phi(\boldsymbol{X})\phi(\boldsymbol{X})^t$$



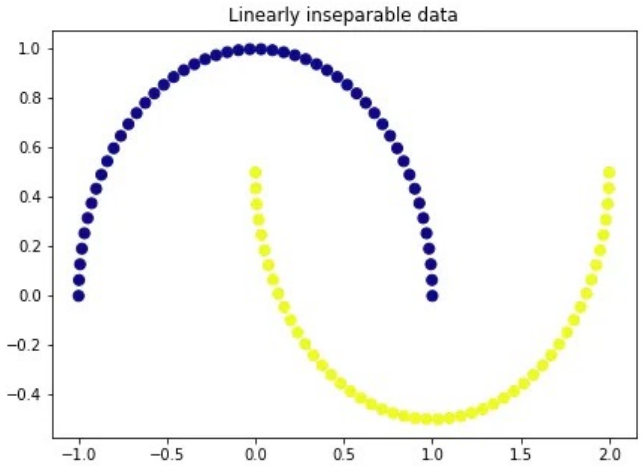
Kernel trick

- For K=3, computations with $\boldsymbol{Z}$ is OK
- For large K, e.g. Radial Basis Function (K=∞), explicit use of $\boldsymbol{Z}$ is not feasible
- $\kappa\left(x^{(i)}, x^{(j)}\right) = \left\langle \phi\left(x^{(i)}\right), \phi\left(x^{(j)}\right) \right\rangle$ is sometimes easy to calculate directly

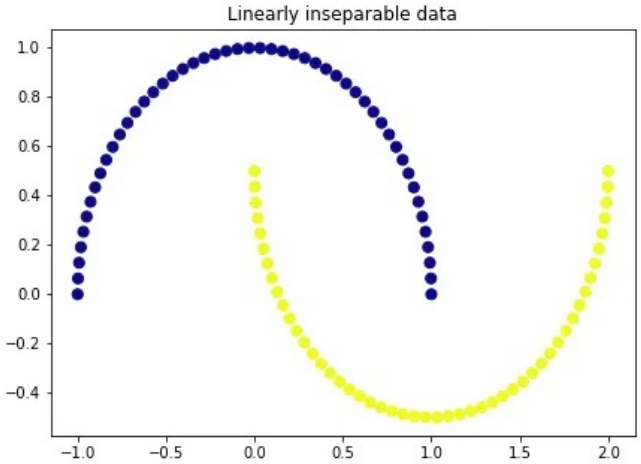
- Solution: work with similarity defined through a Kernel matrix:

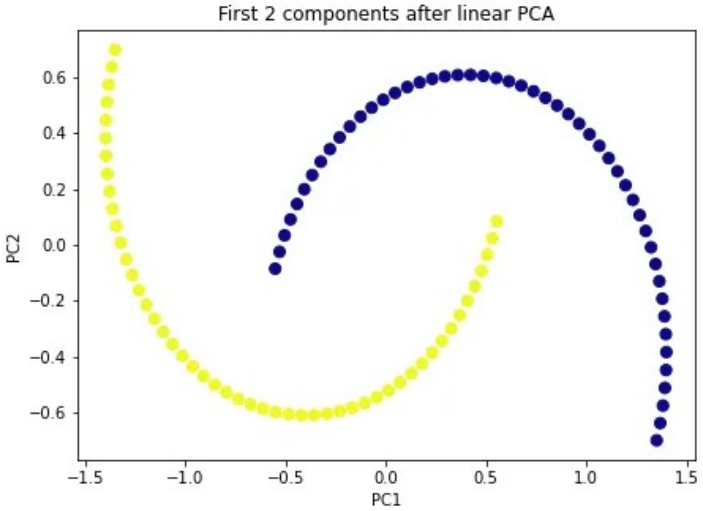
$$\boldsymbol{K} = \begin{bmatrix} \kappa\left(\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(1)}\right) & \kappa\left(\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}\right) & \cdots & \kappa\left(\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(n)}\right) \\ \kappa\left(\boldsymbol{x}^{(2)}, \boldsymbol{x}^{(1)}\right) & \left(\boldsymbol{x}^{(2)}, \boldsymbol{x}^{(2)}\right) & \cdots & \kappa\left(\boldsymbol{x}^{(2)}, \boldsymbol{x}^{(n)}\right) \\ \vdots & \vdots & \ddots & \vdots \\ \kappa\left(\boldsymbol{x}^{(n)}, \boldsymbol{x}^{(1)}\right) & \kappa\left(\boldsymbol{x}^{(n)}, \boldsymbol{x}^{(2)}\right) & \cdots & \kappa\left(\boldsymbol{x}^{(n)}, \boldsymbol{x}^{(n)}\right) \end{bmatrix}$$

Working with kernels

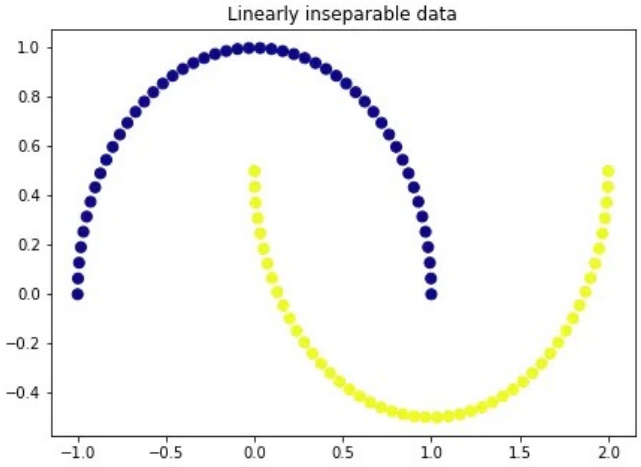


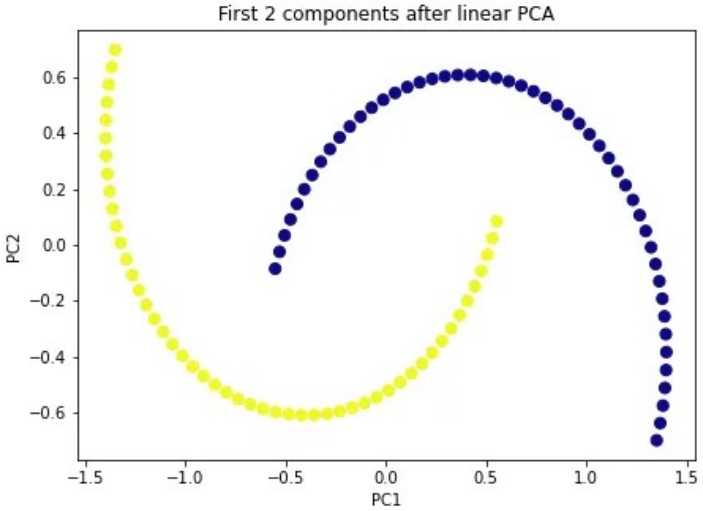
Working with kernels

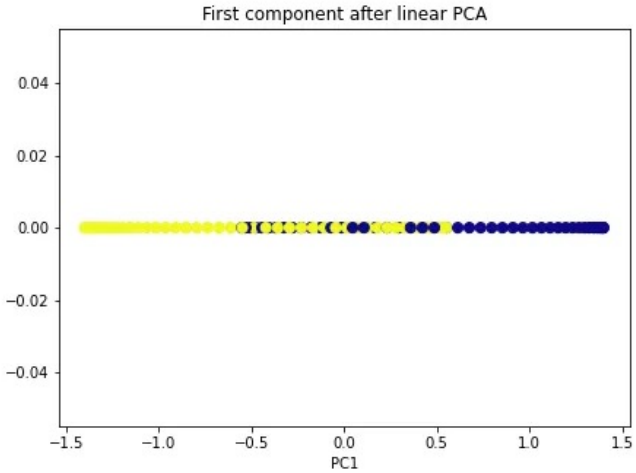




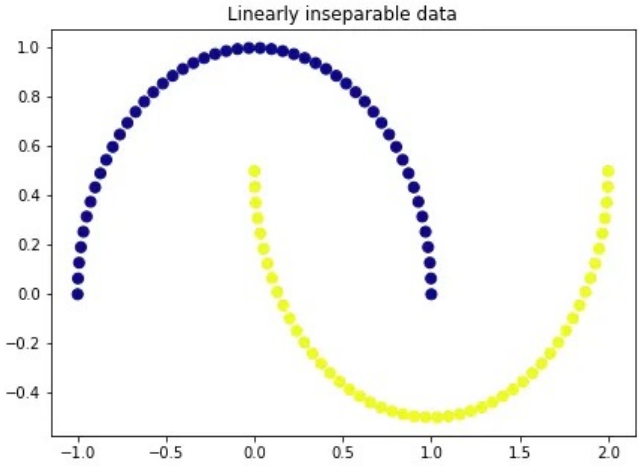
Working with kernels

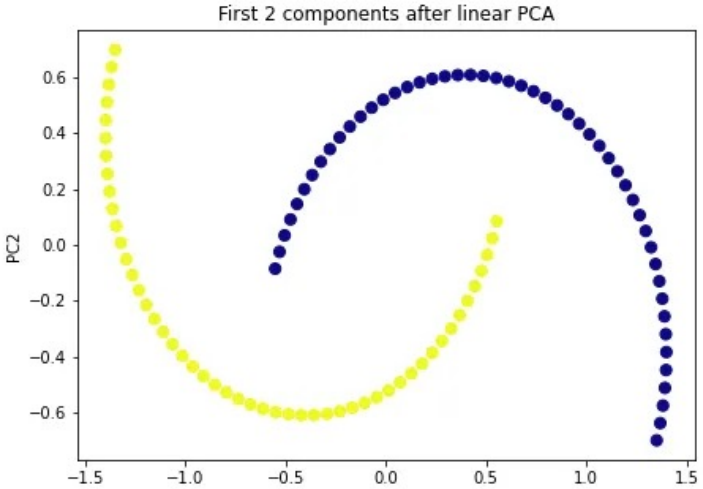


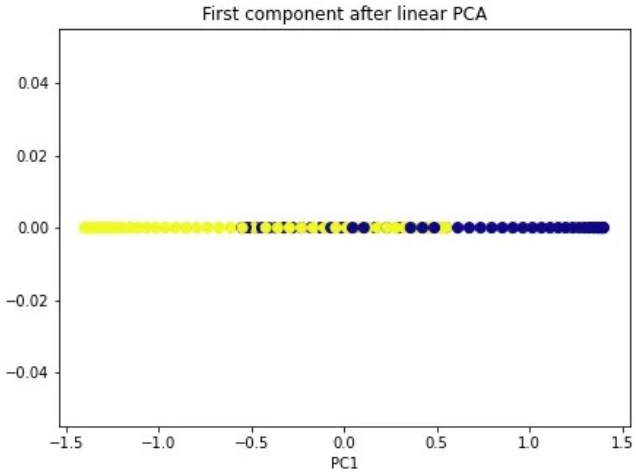


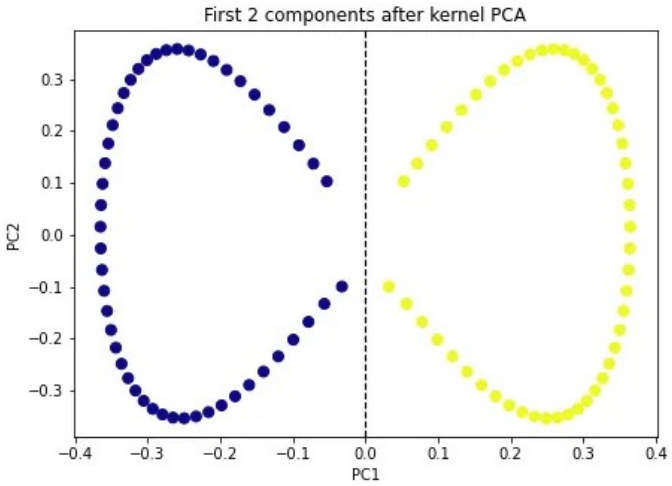


Working with kernels

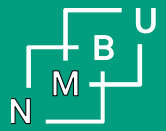








Jupyter: Chapter_5_LDA_Kernel_PCA.ipynb


U
B
M
N